

阿飄單車 (APIOBike)

阿飄單車 (APIOBike) 是阿飄城 (APIO City) 提供的一項新的單車共享服務，城市中設有諸多的租借站，提供使用者借用或歸還阿飄單車。由於收費低廉且便利，阿飄單車很快成為阿飄城中最重要的大眾運輸工具。然而，阿飄單車行在營運上，遇到所有單車共享業者共同的問題：在不同租借站裡，單車借用與歸還的數量無法平衡。這導致某些租借站只有極少的單車可借用，因此許多附近住戶借不到單車；相對的，某些租借站則堆滿了大量沒人要借的單車。

為了克服這個問題，阿飄單車行準備派遣貨車來平衡供需，每日深夜將會以貨車重新分配各租借站的單車數量，藉以確保每個租借站都有足夠數量的單車可用。阿飄城中，一共設有 N 個租借站，並以編號 $0, 1, \dots, N - 1$ 表示。透過觀察，單車行發現每晚停在出租站 i 的單車數量總是 $A[i]$ ，並且不會有人在深夜時租還單車；公司期待每天清早租借站 i 裡能恰好有 $B[i]$ 台單車。

這 N 間租借站間，共有 $N - 1$ 條專用道路讓這輛平衡供需的貨車使用；每條專用道路連接兩個相異的租借站，而且貨車只能使用這些專用道路來運送單車。路的長度皆為一單位長，並且所有的租借站靠專用道路連接在一起，換言之，貨車可以透由專用道路通往任何兩個租借站。每天深夜，阿飄單車行會派出貨車，確保租借站 i 能剛好有 $B[i]$ 台單車。貨車可以從任何一個租借站出發，並且完成任務後停留在任何一個租借站。

執行任務時，貨車一開始是空車。當貨車抵達某個租借站時，司機可以從租借站裝載任意數量的單車到貨車上、或是卸下貨車上任意數量的單車到租借站。貨車或是租借站沒有單車容量的限制，當然租借站或是貨車不能有負的數量。單車行想知道能平衡單車供需且使得貨車移動距離最短的作法。

實作細節 (Implementation Details)

你需要實作出下列程序。

```
std::pair<std::vector<int>, std::vector<long long>>
find_rebalancing_strategy(int N,
                           std::vector<int> A,
                           std::vector<int> B,
                           std::vector<int> U,
                           std::vector<int> V)
```

- A ：長度為 N 的陣列，其中 $A[i]$ 為租借站 i 每天傍晚時分的單車數量。
- B ：長度為 N 的陣列，其中 $B[i]$ 為租借站 i 每天清晨預計的單車數量。

- U 、 V ：長度為 $N - 1$ 的陣列，表示專用道路連通網。對於 $0 \leq i < N - 1$ ，意味者第 i 條路連接租借站 $U[i]$ 及租借站 $V[i]$ 。
- 針對每筆測資，此程序**至多被呼叫** 150 000 次。

此程序應回傳一組長度皆為 $k + 1$ 的陣列 (X, Y) ，用來表示平衡單車供需的策略：

- X ：貨車依序抵達的租借站的編號
- Y ：所對應的租借站的貨車裝卸單車的數量。對於每個 $0 \leq j \leq k$ ：
 - 如果 $Y[j] \geq 0$ ，那麼司機將會從貨車上卸下 $Y[j]$ 台單車到租借站 $X[j]$ ，這會使得此站的單車數量增加 $Y[j]$ 。
 - 如果 $Y[j] < 0$ ，那麼司機將會從租借站 $X[j]$ 裝載 $-Y[j]$ 台單車到貨車上，這會使得此站的單車數量減少 $-Y[j]$ 。

一個**合法**的平衡策略 (X, Y) 且移動距離為 k ，必須滿足下列條件：

- 對於每個 $0 \leq j \leq k$ ，滿足 $0 \leq X[j] < N$ 。
- 對於每個 $0 \leq j < k$ ，租借站 $X[j]$ 與租借站 $X[j + 1]$ 必須直接被一條專用道路連接。
- 對於每個 $0 \leq j \leq k$ ，滿足 $\sum_{t=0}^j Y[t] \leq 0$ 。
- 對於每個租借站 $0 \leq i < N$ 及每個步驟 $0 \leq j \leq k$ ，令 $S_{i,j}$ 為針對所有的 t ，滿足 $0 \leq t \leq j$ 且 $X[t] = i$ ，如此的 $Y[t]$ 的和。
 - 租借站 i 的單車數量不會少於零： $A[i] + S_{i,j} \geq 0$ 。
 - 平衡策略施行後，每個租借站 i 必須恰好有 $B[i]$ 台單車： $A[i] + S_{i,k} = B[i]$ 。

限制條件 (Constraints)

- $2 \leq N \leq 300\,000$
- 每筆測資中，所有呼叫 `find_rebalancing_strategy` 所使用的 N 的總和不超過 300 000。
- 對於每個 $0 \leq i < N$ ，皆有 $0 \leq U[i], V[i] < N$ 且 $U[i] \neq V[i]$ 。
- 對於每個 $0 \leq i < N$ ，皆有 $0 \leq A[i], B[i] \leq 10^9$ 。
- 任兩個租借站皆可往返。
- $\sum_{i=0}^{N-1} A[i] = \sum_{i=0}^{N-1} B[i]$ 。
- 至少有一個 i 滿足 $0 \leq i < N$ 且 $A[i] \neq B[i]$ 。

子任務與得分 (Subtasks and Scoring)

此處 T 為呼叫 `find_rebalancing_strategy` 的總次數， $\sum N$ 為所有呼叫 `find_rebalancing_strategy` 所使用的 N 的總和。

子任務	得分	額外限制
1	4	租借站與道路滿足性質 P (如下述)。 僅有一個 i 滿足 $0 \leq i < N$ 且 $A[i] > 0$ 。
2	11	$T \leq 10, N \leq 7$ 。租借站與道路滿足性質 P。
3	16	租借站與道路滿足性質 P。
4	9	僅有一個 i 滿足 $0 \leq i < N$ 且 $A[i] > 0$ 。
5	24	$\sum N \leq 500$ 。
6	15	$\sum N \leq 5000$ 。
7	21	無額外限制。

性質 P：對於每個 $0 \leq i < N - 1$ ，滿足 $U[i] = i$ 且 $V[i] = i + 1$ 。對於每個 $0 \leq i < N$ ，滿足 $A[i] \neq B[i]$ 。

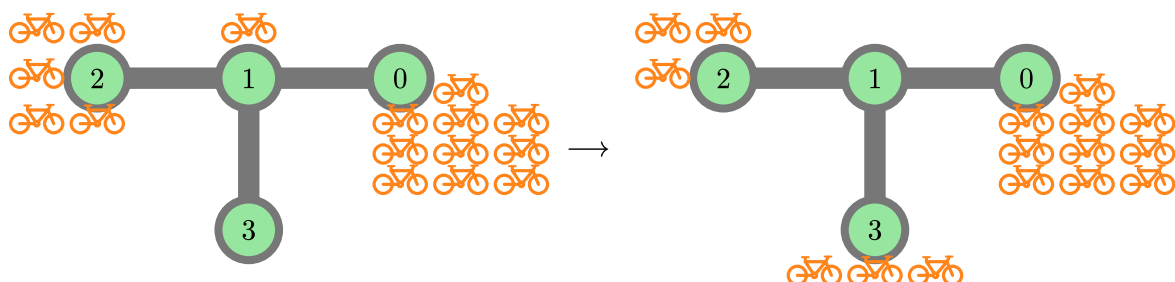
如果回傳的陣列 X 與 Y 的長度恰好是 $k^* + 1$ ，其中 k^* 為可能的最小移動距離，縱使 (X, Y) 並非合法策略，你將會獲得 50% 的得分。

範例 (Example)

範例 1

考慮下列呼叫：

```
find_rebalancing_strategy(4,
    [10, 1, 5, 0],
    [10, 0, 3, 3],
    [0, 1, 1],
    [1, 2, 3])
```



阿飄城中共有 $N = 4$ 個租借站。一開始租借站各有 $A = [10, 1, 5, 0]$ 台單車，目標是使得租借站各有 $B = [10, 0, 3, 3]$ 台單車。

假設貨車從租借站 2 出發，並且依循下列步驟：

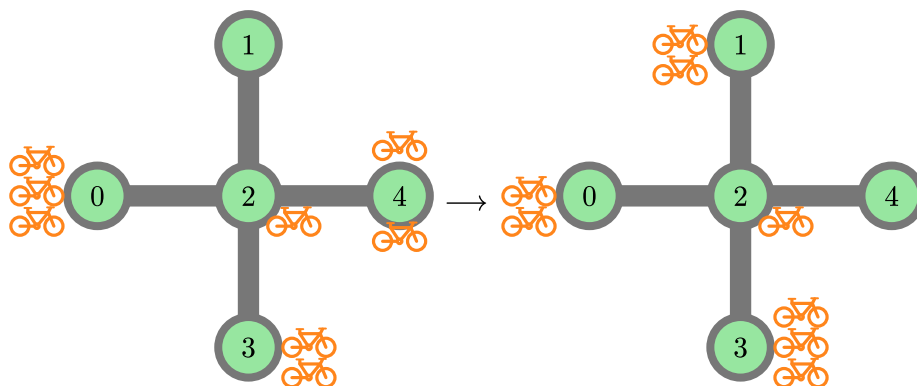
- 在租借站 2 裝載 2 台單車到貨車上。此時租借站 2 有 3 台單車，貨車上有 2 台單車。
- 移動到租借站 1 並且裝載 1 台單車到貨車上。此時租借站 1 有 0 台單車，貨車上有 3 台單車。
- 移動到租借站 3 並且卸下 3 台單車。此時租借站 3 有 3 台單車，貨車上有 0 台單車。

總移動距離為 2。對應此策略的回傳陣列為 $X = [2, 1, 3]$ 與 $Y = [-2, -1, 3]$ 。不難證明此策略獲得最短移動距離，因此程序應當回傳 $([2, 1, 3], [-2, -1, 3])$ 。

範例 2

考慮下列呼叫：

```
find_rebalancing_strategy(5,  
                           [3, 0, 1, 2, 2],  
                           [2, 2, 1, 3, 0],  
                           [2, 2, 2, 2],  
                           [0, 4, 3, 1])
```



阿飄城中共有 $N = 5$ 個租借站。一開始租借站各有 $A = [3, 0, 1, 2, 2]$ 台單車，目標是使得租借站各有 $B = [2, 2, 1, 3, 0]$ 台單車。

假設貨車從租借站 0 出發，並且依循下列步驟：

- 在租借站 0, 裝載 1 台單車。
- 移動到租借站 2 並且裝載 1 台單車。
- 移動到租借站 1 並且卸下 2 台單車。
- 移動到租借站 2。
- 移動到租借站 4 並且裝載 2 台單車。
- 移動到租借站 2 並且卸下 1 台單車。

- 移動到租借站 3 並且卸下 1 台單車。

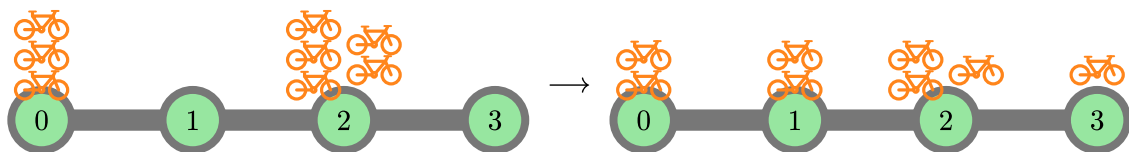
總移動距離為 6。對應此策略的回傳陣列為 $X = [0, 2, 1, 2, 4, 2, 3]$ 與 $Y = [-1, -1, 2, 0, -2, 1, 1]$ 。不難證明此策略獲得最短移動距離，因此程序應當回傳 $([0, 2, 1, 2, 4, 2, 3], [-1, -1, 2, 0, -2, 1, 1])$ 。

其他同樣距離為 6 的合法策略，如 $X = [0, 2, 3, 2, 4, 2, 1]$ 與 $Y = [-1, 0, 1, 0, -2, 0, 2]$ ，皆為正確答案。若回傳陣列 X 、 Y 長度同樣為 $7 = 6 + 1$ ，但策略並不合法，如 $X = Y = [0, 0, 0, 0, 0, 0, 0]$ ，將會獲得 50% 的得分。

範例 3

考慮下列呼叫：

```
find_rebalancing_strategy(4,
                          [3, 0, 5, 0],
                          [2, 2, 3, 1],
                          [0, 1, 2],
                          [1, 2, 3])
```



最佳解可為 $X = [2, 1, 0, 1, 2, 3]$ 與 $Y = [-1, 1, -1, 1, -1, 1]$ 。程序應該回傳 $([2, 1, 0, 1, 2, 3], [-1, 1, -1, 1, -1, 1])$ 。其他的合法策略，如 $X = [2, 1, 0, 1, 2, 3]$ 與 $Y = [-2, 2, -1, 0, 0, 1]$ ，皆視為正確答案。

此範例滿足子任務 2 與 3 的限制。

範例評分程式 (Sample Grader)

輸入第一行包含單一整數 T ，表示總情境數。接著會有 T 個情境的描述，格式如下。

輸入格式：

```
N
A[0] A[1] ... A[N-1]
B[0] B[1] ... B[N-1]
U[0] V[0]
U[1] V[1]
...
U[N-2] V[N-2]
```

輸出格式：

```
k
X[0] X[1] ... X[k]
Y[0] Y[1] ... Y[k]
```